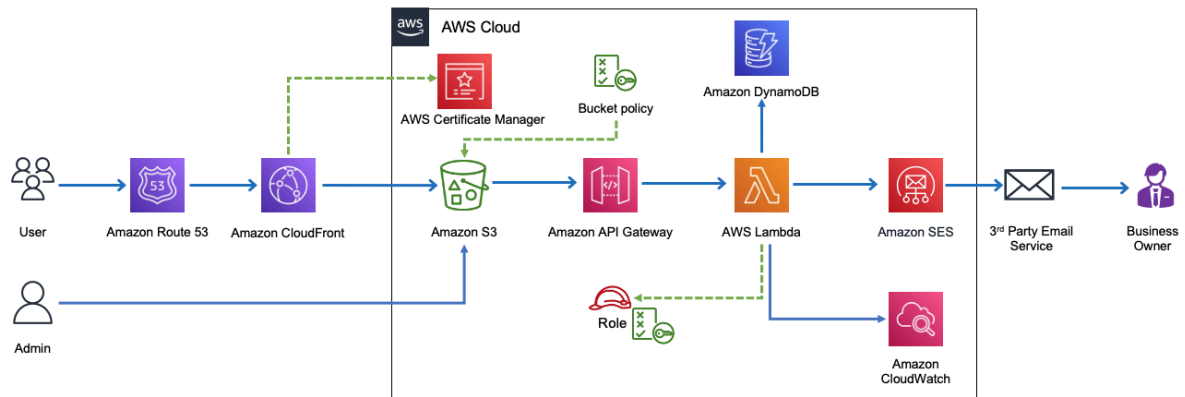
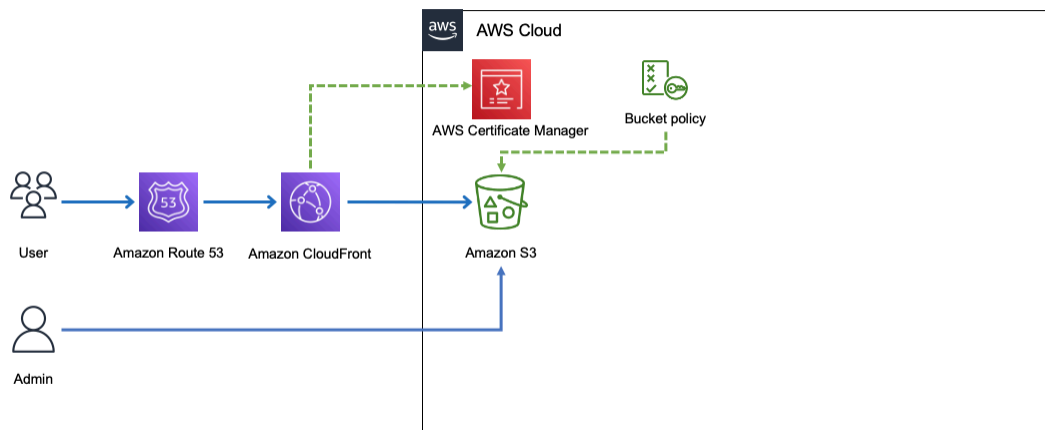


Build a Serverless Lead Capture on AWS



Phase 1: Hosting Static Website



STEP 1: Download website

You can clone project form my Github:

```
git clone https://github.com/pravinmishraaws/Ebook.git
```

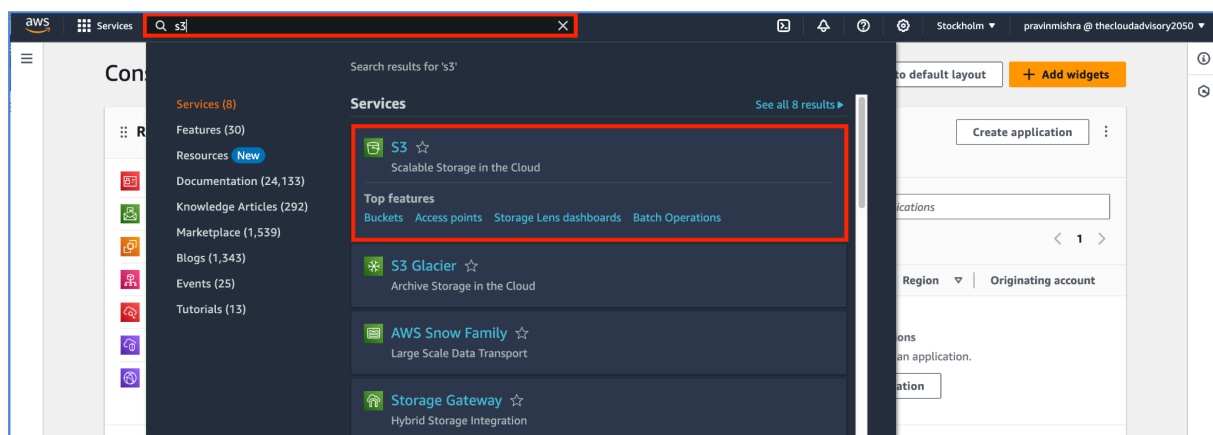
STEP 2: Locate the downloaded file in your terminal

```
ls
```

STEP 3: Create Amazon S3 bucket

```
aws s3 ls
aws s3 mb s3://epicreads
```

STEP 4: Edit static website hosting



STEP 5: Edit bucket settings

403 Forbidden

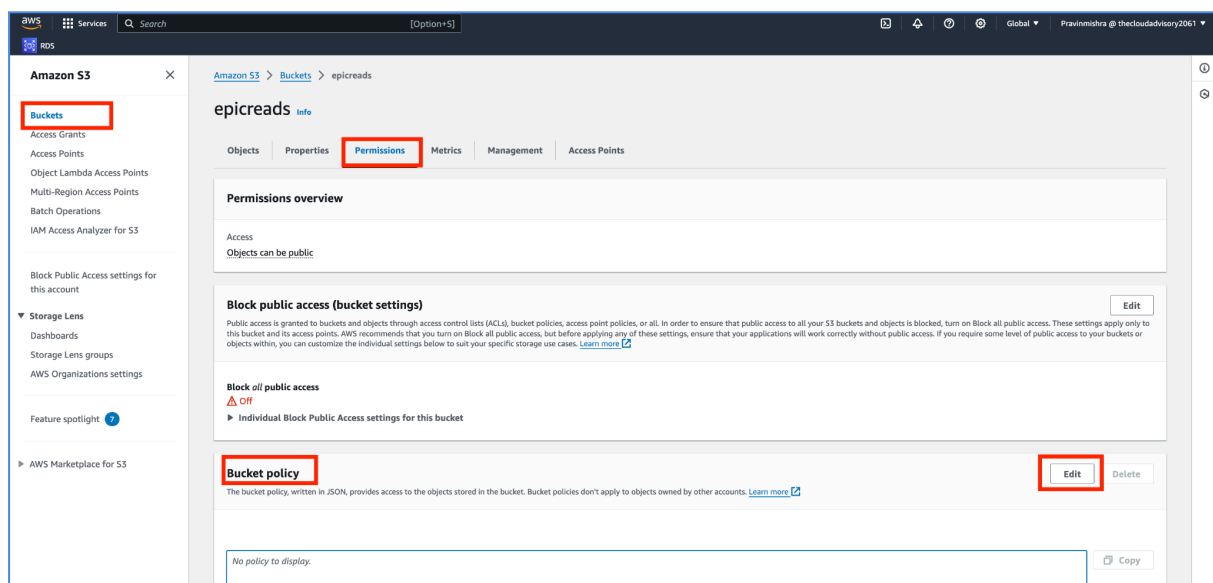
- Code: AccessDenied
- Message: Access Denied
- RequestId: 64086Z8TKE2QYTRY
- HostId: 2fFPQsUdBKspZUhWMiK48k1GCB4Y0b/JyogWaHry06MEhwLp/9FiO/bmMIH7rTs3irVa2a6bH0g=

An Error Occurred While Attempting to Retrieve a Custom Error Document

- Code: AccessDenied
- Message: Access Denied

STEP 6: Add Bucket policy

Note: Multi Level of protection.



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
```

```

        "Principal": "*",
        "Action": [
            "s3:GetObject"
        ],
        "Resource": [
            "arn:aws:s3:::Bucket-Name/*"
        ]
    }
}

```

404 Not Found

- Code: NoSuchKey
- Message: The specified key does not exist.
- Key: index.html
- RequestId: 5QKBFJ8QEYZTFDGF
- HostId: TWgH+4ZNhRTj2f11pYINgDKpA3CGTFZUIP9GN/uRXiFWgnA8FLIsiQfjqK/LzvJB51+XYkofoFI=

An Error Occurred While Attempting to Retrieve a Custom Error Document

- Code: NoSuchKey
- Message: The specified key does not exist.
- Key: 404.html

STEP 7: Display error page

```
vi 404.html
```

Hi I am an error page!

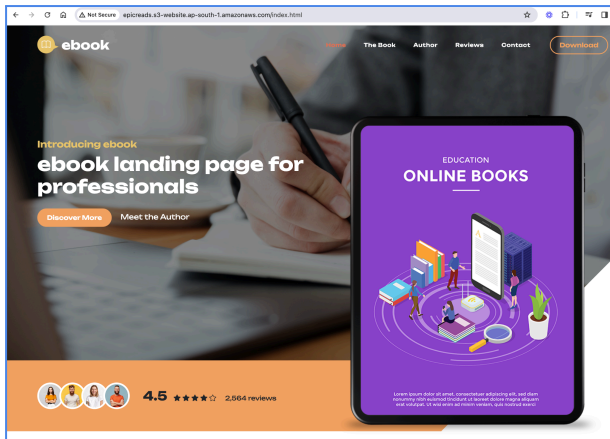
Sorry, I will be back soon!

```
aws s3 cp 404.html s3://epicreads
```

STEP 8: Sync the files to S3 bucket

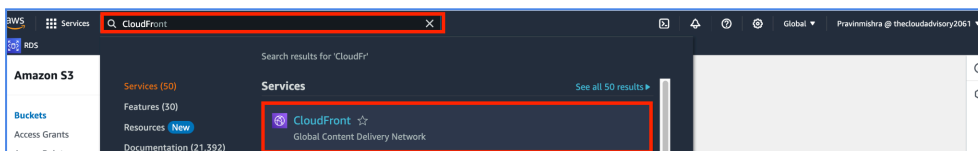
```
aws s3 sync . s3://epicreads
```

STEP 9: Refresh the browser



STEP 10: Public Certificate (us east)

STEP 11: Configure CloudFront (global)

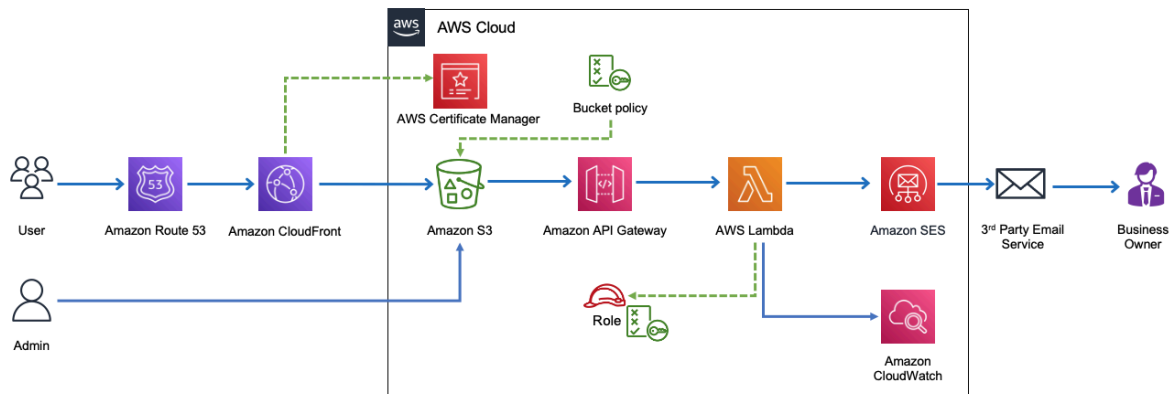


Congrats you have successfully completed Project 9!

STEP 12: Configure Route 53

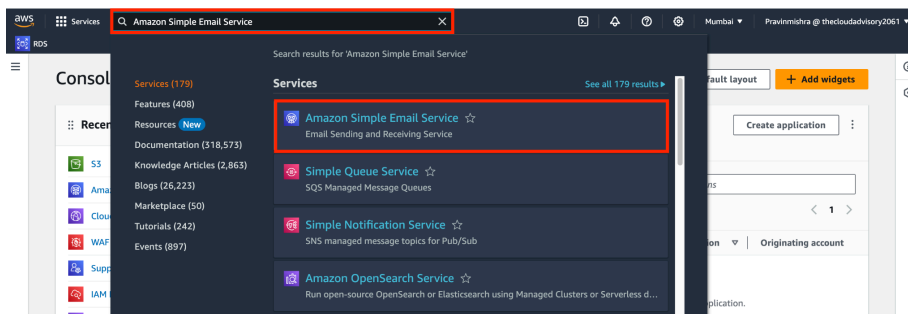
Phase 2: Implementing a Dynamic 'Contact Us'

STEP 1: Understand the architecture



STEP 2: Understand the structure of 'Contact Us' Form

STEP 3: Setup Amazon SES



STEP 4: Create IAM Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CloudWatchLogsBasic",
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup"],
      "Resource": "*"
    }
  ]
}
```

```

    },
    {
      "Sid": "CloudWatchLogsStreamEvents",
      "Effect": "Allow",
      "Action": ["logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "*"
    },
    {
      "Sid": "AllowSesSendEmail",
      "Effect": "Allow",
      "Action": ["ses:SendEmail", "ses:SendRawEmail"],
      "Resource": "*"
    }
  ]
}

```

Policy Name:

Epicreads_SES_policy

STEP 5: Create IAM Role

Epicreads_role

STEP 6: Understand AWS Lambda Function

STEP 7: Create Lambda Function

epicreads_contactus

Lambda Code:

```

import { SESClient, SendEmailCommand } from "@aws-sdk/client-ses";

const ses = new SESClient({ region: "us-east-1" });

const RECEIVER = "hello@pravinmishra.in";
const SENDER = "mail@pravinmishra.in";

export const handler = async (event) => {
  console.log("Received event:", event);

  const params = {
    Destination: { ToAddresses: [RECEIVER] },
    Message: {
      Body: {

```

```

        Text: {
            Data: `Full Name: ${event.name}
Phone: ${event.phone}
Email: ${event.email}
Message: ${event.message}` ,
            Charset: "UTF-8",
        },
    },
    Subject: { Data: `Website Query Form: ${event.name}`, Charset: "UTF-8" },
},
Source: SENDER,
};

await ses.send(new SendEmailCommand(params));

return {
    statusCode: 200,
    headers: {
        "Content-Type": "application/json",
        "Access-Control-Allow-Origin": "*",
    },
    body: JSON.stringify({ result: "Success" }),
};
};

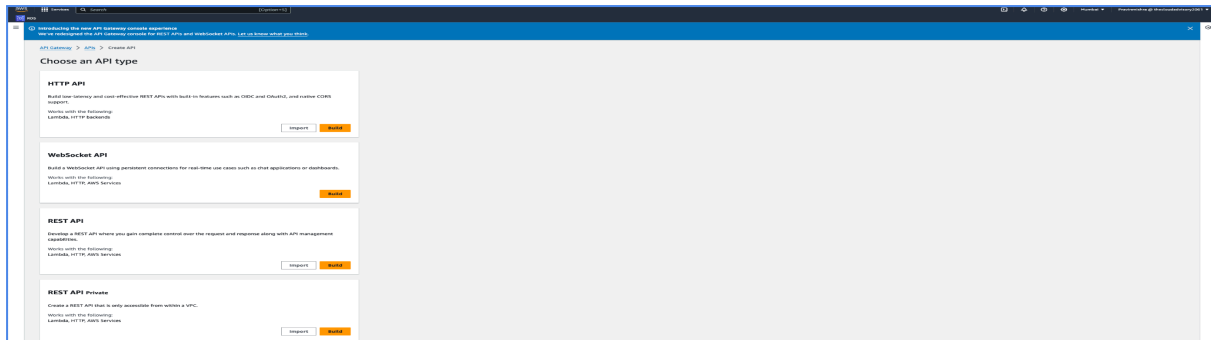
```

STEP 8: Test the function in the console

Name the event as "TestEvent".

STEP 9: Build an API

Understanding APIs



HTTP API:

- This type of API is designed for building low-latency and cost-effective RESTful APIs (Representational State Transfer) for web applications.
- It comes with built-in features like OIDC (OpenID Connect) and OAuth2 for handling authentication and authorization.
- It also has native CORS (Cross-Origin Resource Sharing) support, which allows web applications running at one origin to request resources from another origin securely.
- It works seamlessly with AWS Lambda functions and HTTP backends, making it easy to integrate with your serverless or traditional backend services.

WebSocket API:

- WebSocket APIs are used for building real-time communication applications like chat applications, online gaming, or dashboards that require persistent and bidirectional connections.
- These APIs are designed for handling WebSocket connections, enabling real-time data exchange between clients and servers.
- WebSocket APIs can work with various backend services, including AWS Lambda functions, HTTP endpoints, and AWS services, allowing you to build interactive and responsive applications.

REST API:

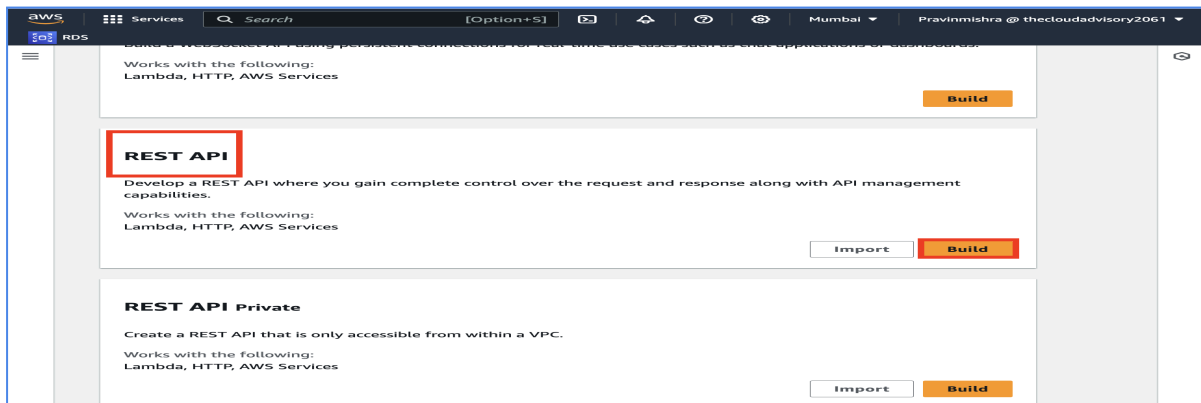
- REST APIs are a standard way to develop web services following the principles of Representational State Transfer (REST).
- With a REST API, you have complete control over the request and response structure, making it highly customizable to suit your application's needs.

- REST APIs can be integrated with various backend services, including AWS Lambda functions, HTTP endpoints, and AWS services, giving you flexibility in designing and managing your APIs.

REST API Private:

- This is a variation of a REST API designed to be accessible only from within an Amazon Virtual Private Cloud (VPC).
- It provides an extra layer of security by ensuring that the API is isolated from the public internet and can only be accessed by resources within your VPC.
- Like the standard REST API, it can also work with various backend services, including AWS Lambda functions, HTTP endpoints, and AWS services.

1. Create an API



API name ***EpicReads_api***

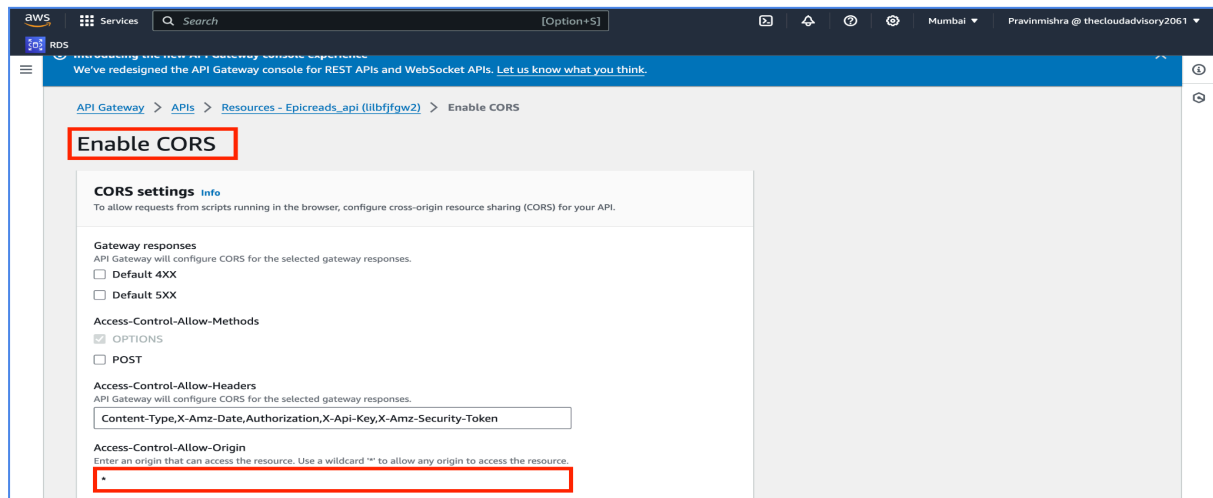
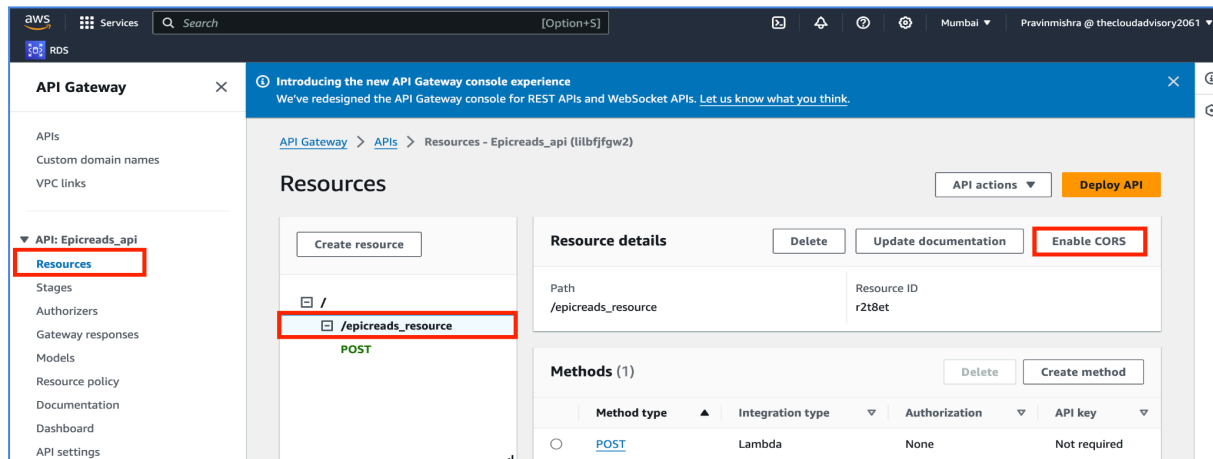
2. Create Resource

Resource Name ***epicreads_resource***

3. Create Method

Choose **Post** for *Method type*.

STEP 10: Enable CORS



STEP 11: Deploy API

Deploy API

Choose a stage where your API will be deployed. For example, a test version of your API could be deployed to a stage named beta.

Stage
New stage

Stage name
prod

i A new stage will be created with the default settings. Edit your stage settings on the **Stage** page.

Deployment description
prod

Cancel Deploy

STEP 12: Test API with CURL

1) Preflight (OPTIONS) — simulate the browser check

```
curl -i -X OPTIONS
"https://rp17fhoig9.execute-api.us-east-1.amazonaws.com/dev/epicread
s_resource" \
-H "Origin: https://your-site-domain.com" \
-H "Access-Control-Request-Method: POST" \
-H "Access-Control-Request-Headers:
Content-Type,Authorization,X-API-Key"
```

You should see 200 and headers like:

```
Access-Control-Allow-Origin, Access-Control-Allow-Methods,
Access-Control-Allow-Headers.
```

2) Actual POST request (JSON body)

```
curl -i -X POST
"https://rp17fhoig9.execute-api.us-east-1.amazonaws.com/dev/epicread
s_resource" \
-H "Origin: https://your-site-domain.com" \
-H "Content-Type: application/json" \
-d '{
    "name": "Pravin Mishra",
```

```
"phone": "+358-555-1234",
"email": "user@example.com",
"message": "Hello from curl"
}'
```

STEP 13: Update Website

Update the form with below inside the index.html:

```
<form id="contact-form" class="custom-form ebook-download-form bg-white shadow"
role="form">
```

Javascript:

```
<script>
  (function () {
    const URL =
"https://j5onc24oq0.execute-api.eu-north-1.amazonaws.com/dev/epicreads_resource";

    const form = document.getElementById('contact-form');
    if (!form) {
      console.error("contact-form not found in DOM");
      return;
    }

    form.addEventListener('submit', async function (e) {
      e.preventDefault();

      const name = document.getElementById('ebook-form-name').value ||
"";

      const email = document.getElementById('ebook-email').value || "";
      const messageEl = document.getElementById('ebook-message'); //
optional field
      const message = messageEl ? messageEl.value : "";

      try {
        const resp = await fetch(URL, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json', 'Accept':
'application/json' },
          body: JSON.stringify({ name, email, message }),
          // mode: 'cors' // not required, but harmless if you want to
make it explicit
```

```

    });

    const contentType = resp.headers.get('content-type') || '';
    let bodyText = '';
    try { bodyText = await resp.text(); } catch { }

    console.log('API status:', resp.status);
    console.log('API headers:',
Object.fromEntries(resp.headers.entries()));
    console.log('API body:', bodyText);

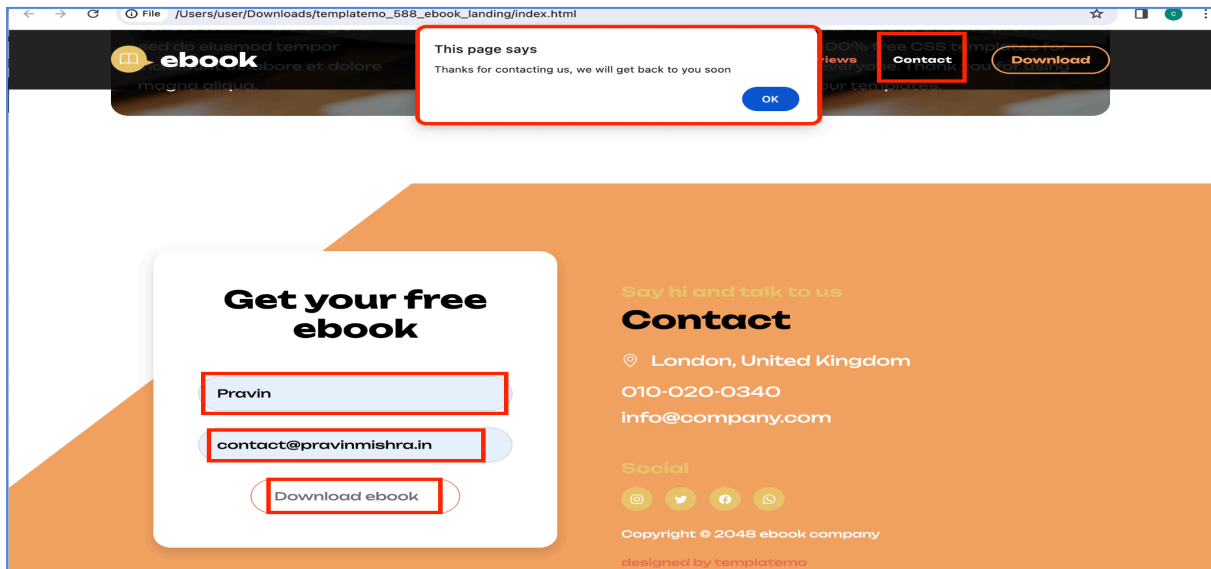
    if (!resp.ok) {
        throw new Error(`HTTP ${resp.status} - ${bodyText || 'No
body'} `);
    }

    alert("✅ Thanks for contacting us, we will get back to you
soon!");

    this.reset();
} catch (err) {
    console.error("Request failed:", err);
    alert("❌ Something went wrong. Please try again.");
}
});
})();
</script>

```

STEP 14: Test Locally



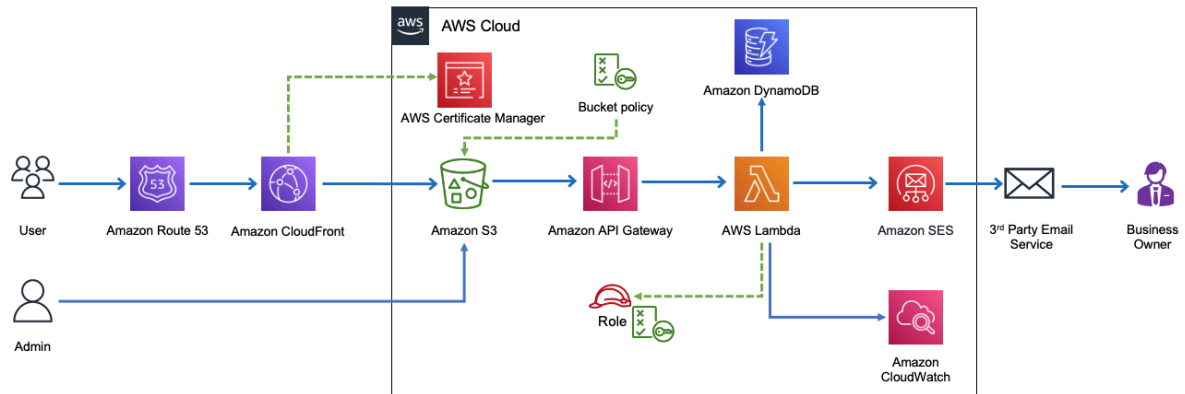
STEP 15: Upload new changes to S3 & Test

`aws s3 sync . s3://epicreads`

STEP 16: Test with domain name

STEP 17: Verify CloudWatch Log

Phase 3: Data Management for 'Contact Us'



STEP 1: Create a table

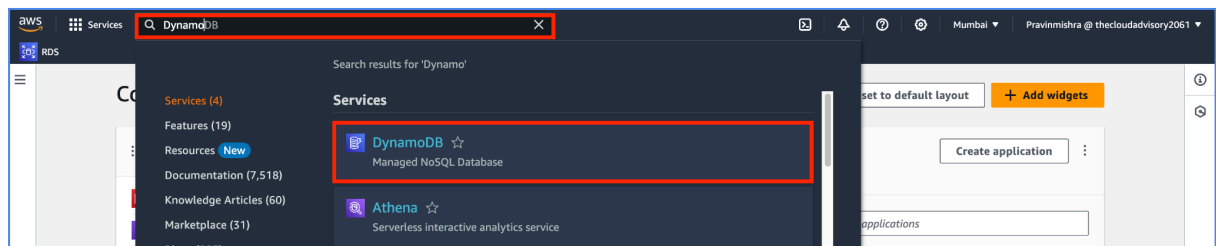


Figure 12.1: Navigate to DynamoDB

Table name, enter **ContactMessages**.
For the Partition key name, enter **id**.
For the Partition key type, choose **String**.

Create the table (one-time) with AWS CLI

```
aws dynamodb create-table \  
  --table-name ContactMessages \  
  --attribute-definitions AttributeName=id,AttributeType=S \  
  --key-schema AttributeName=id,KeyType=HASH \  
  --billing-mode PAY_PER_REQUEST \  
  --region us-east-1
```


STEP 2: Update Lambda Role Permission

Create Inline Policy (LambdaContact-DdbWrite)

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": ["dynamodb:PutItem"],
      "Resource": "*" }
  ]
}
```

STEP 3: Update the Lambda function code

Use below updated Lambda code that writes to **DynamoDB** and still sends the email via **SES**.

```
import { SESClient, SendEmailCommand } from "@aws-sdk/client-ses";
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, PutCommand } from
"@aws-sdk/lib-dynamodb";
import { randomUUID } from "crypto";

const REGION = process.env.AWS_REGION || "us-east-1";           // keep
SES & DDB in same region
const TABLE_NAME = process.env.TABLE_NAME || "ContactMessages";

const RECEIVER = "hello@pravinmishra.in";
const SENDER    = "mail@pravinmishra.in";

const ses = new SESClient({ region: REGION });
const ddb = DynamoDBDocumentClient.from(new DynamoDBClient({ region:
REGION }));

const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

export const handler = async (event) => {
```

```

    console.log("Received event:", JSON.stringify(event));

    try {
        const body =
            typeof event?.body === "string" ? JSON.parse(event.body) :
            (event?.body ?? event ?? {});

        const { name = "", phone = "", email = "", message = "" } =
body;

        if (!name || !email) {
            return {
                statusCode: 400,
                headers: corsHeaders(),
                body: JSON.stringify({ error: "name and email are required"
            })
        };
    }

    const id = randomUUID();
    const ts = new Date().toISOString();

    // 1) Save to DynamoDB
    await ddb.send(new PutCommand({
        TableName: TABLE_NAME,
        Item: { id, name, phone, email, message, createdAt: ts }
    }));

    // 2) Send email via SES (only add Reply-To if valid)
    const params = {
        Destination: { ToAddresses: [RECEIVER] },
        Message: {
            Subject: { Data: `Website Query Form: ${name}`, Charset:
"UTF-8" },
            Body: {
                Text: {
                    Data:
`Full Name: ${name}
Phone: ${phone}
Email: ${email}
Message: ${message}

```

```

ID: ${id}
Time: ${ts}`,
    Charset: "UTF-8"
    }
    },
    Source: SENDER
};

if (EMAIL_RE.test(email)) {
    params.ReplyToAddresses = [email];
}

await ses.send(new SendEmailCommand(params));

return {
    statusCode: 200,
    headers: corsHeaders(),
    body: JSON.stringify({ ok: true, id })
};
} catch (err) {
    console.error("Error:", err);
    return {
        statusCode: 500,
        headers: corsHeaders(),
        body: JSON.stringify({ error: "Internal error" })
    };
}
};

function corsHeaders() {
    return {
        "Content-Type": "application/json",
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Methods": "OPTIONS,POST",
        "Access-Control-Allow-Headers":

"Content-Type, Authorization, X-API-Key, X-Amz-Date, X-Amz-Security-Token"
    };
}

```

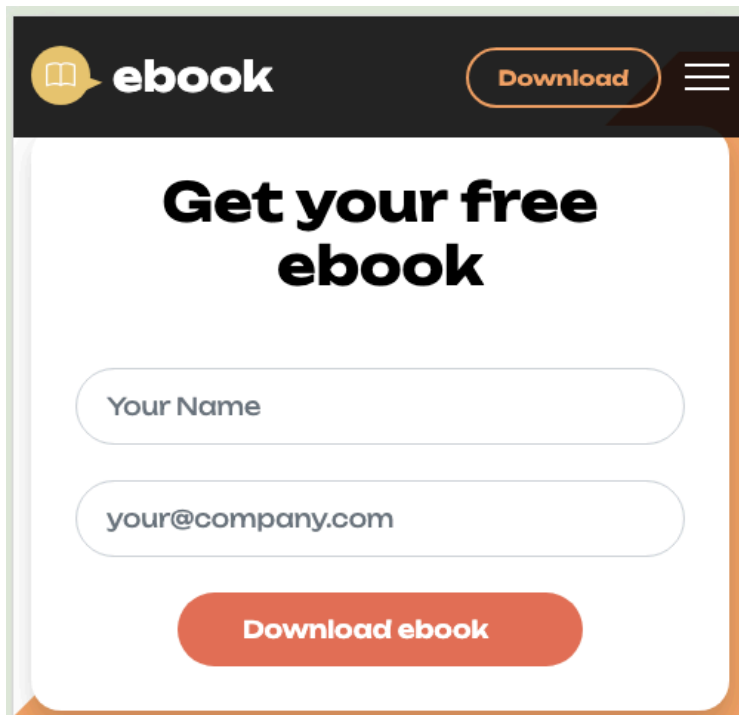
Reminders:

- Your role needs `dynamodb:PutItem` (you said you'll add an inline policy—perfect).
- Create the table `ContactMessages` with hash key `id` (or set `TABLE_NAME` to your table).

STEP 4: Deploy and Test

Verify data in DynamoDB

STEP 5: Test from the browser



The screenshot shows a web interface for downloading a free ebook. At the top, there is a dark header with a logo on the left, a 'Download' button in the center, and a hamburger menu icon on the right. Below the header, the main content area has a white background with an orange border. It features the heading 'Get your free ebook' in bold black text. Underneath the heading are two rounded rectangular input fields: the first is labeled 'Your Name' and the second is labeled 'your@company.com'. At the bottom of the form is a large orange button with the text 'Download ebook' in white.

Task 4: Clean Up!